ST5209/X Assignment 2

Sivakumar Nishaanth

Submission

1. Render the document to get a .pdf printout.
2. Submit both the .qmd and .pdf files to Canvas.

Question 1 (Forecast evaluation, Q5.12 in FPP)

`tourism` contains quarterly visitor nights (in thousands) from 1998 to 2017 for 76 regions of Australia.

```
tourism
```

```
# A tsibble: 24,320 x 5 [1Q]
# Key:       Region, State, Purpose [304]
   Quarter Region   State           Purpose  Trips
     <qtr> <chr>    <chr>           <chr>    <dbl>
 1 1998 Q1 Adelaide South Australia Business  135.
 2 1998 Q2 Adelaide South Australia Business  110.
 3 1998 Q3 Adelaide South Australia Business  166.
 4 1998 Q4 Adelaide South Australia Business  127.
 5 1999 Q1 Adelaide South Australia Business  137.
 6 1999 Q2 Adelaide South Australia Business  200.
 7 1999 Q3 Adelaide South Australia Business  169.
 8 1999 Q4 Adelaide South Australia Business  134.
 9 2000 Q1 Adelaide South Australia Business  154.
10 2000 Q2 Adelaide South Australia Business  169.
# i 24,310 more rows
```

a. Extract data from the Gold Coast region using `filter()` and aggregate total overnight trips using `summarise()`. Call this new dataset `gc_tourism`.

```r
gc_tourism <- tourism |>
  filter(Region == "Gold Coast") |>
  summarise(Total_Trips = sum(Trips))

gc_tourism
```

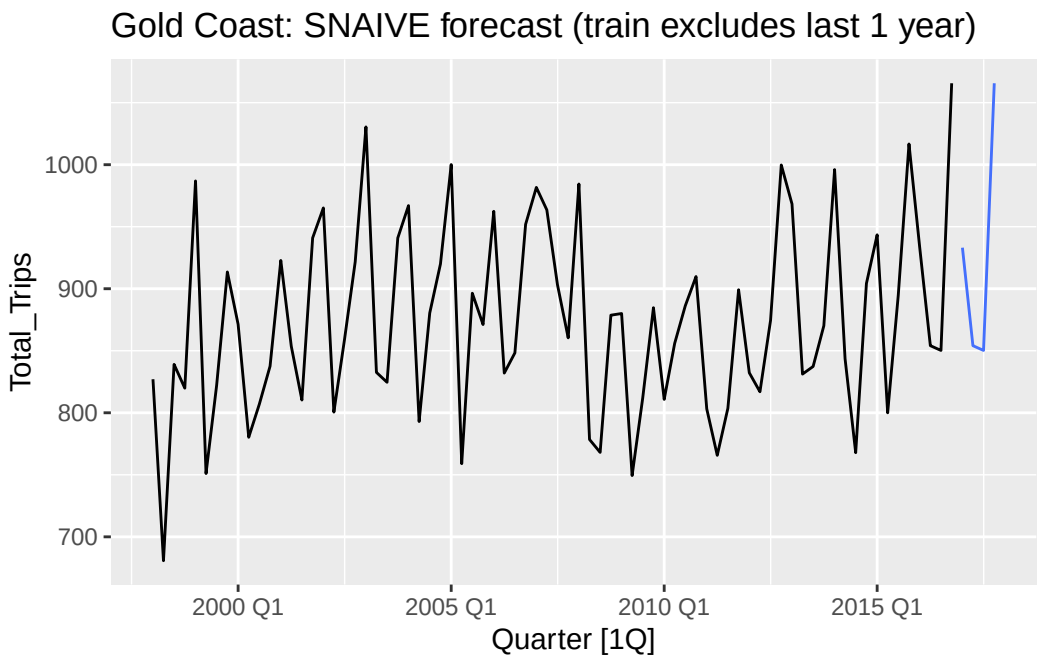
```
# A tsibble: 80 x 2 [1Q]
   Quarter Total_Trips
     <qtr>       <dbl>
 1 1998 Q1        827.
 2 1998 Q2        681.
 3 1998 Q3        839.
 4 1998 Q4        820.
 5 1999 Q1        987.
 6 1999 Q2        751.
 7 1999 Q3        822.
 8 1999 Q4        914.
 9 2000 Q1        871.
10 2000 Q2        780.
# i 70 more rows
```

b. Using `slice()` or `filter()`, create three training sets for this data excluding the last 1, 2 and 3 years. For example, `gc_train_1 <- gc_tourism |> slice(1:(n()-4))`.

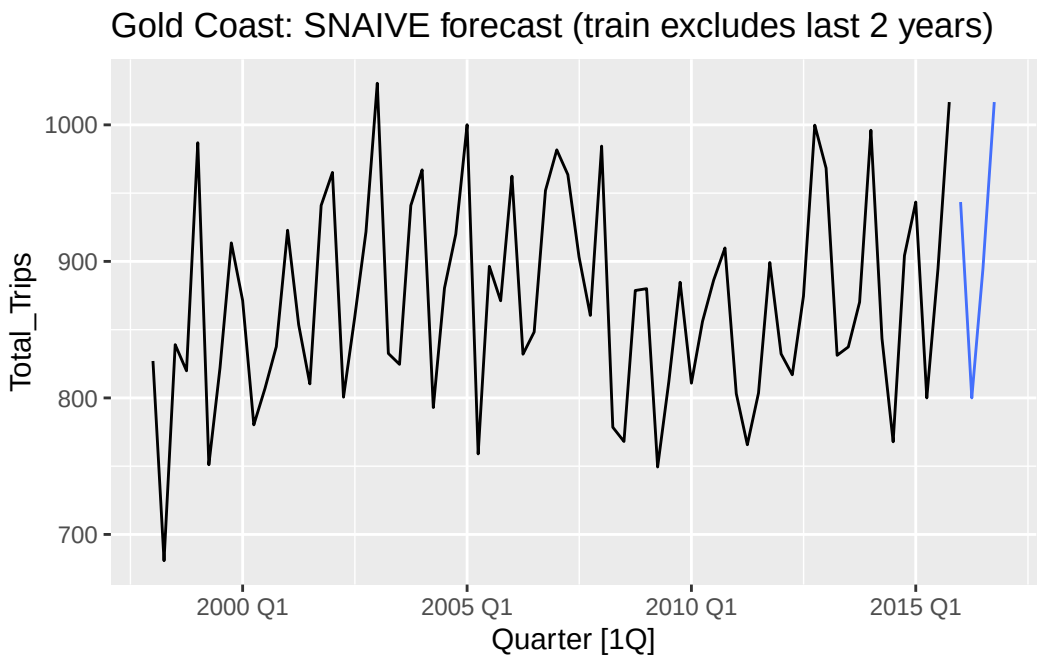
```r
gc_train_1 <- gc_tourism |>
  slice(1:(n() - 4))

gc_train_2 <- gc_tourism |>
  slice(1:(n() - 8))

gc_train_3 <- gc_tourism |>
  slice(1:(n() - 12))
```

c. Compute and plot one year of forecasts for each training set using the seasonal naive (`SNAIVE()`) method. Call these `gc_fc_1`, `gc_fc_2` and `gc_fc_3`, respectively. (Hint: You may combine the mable objects from each model into a single mable using `bind_cols`)

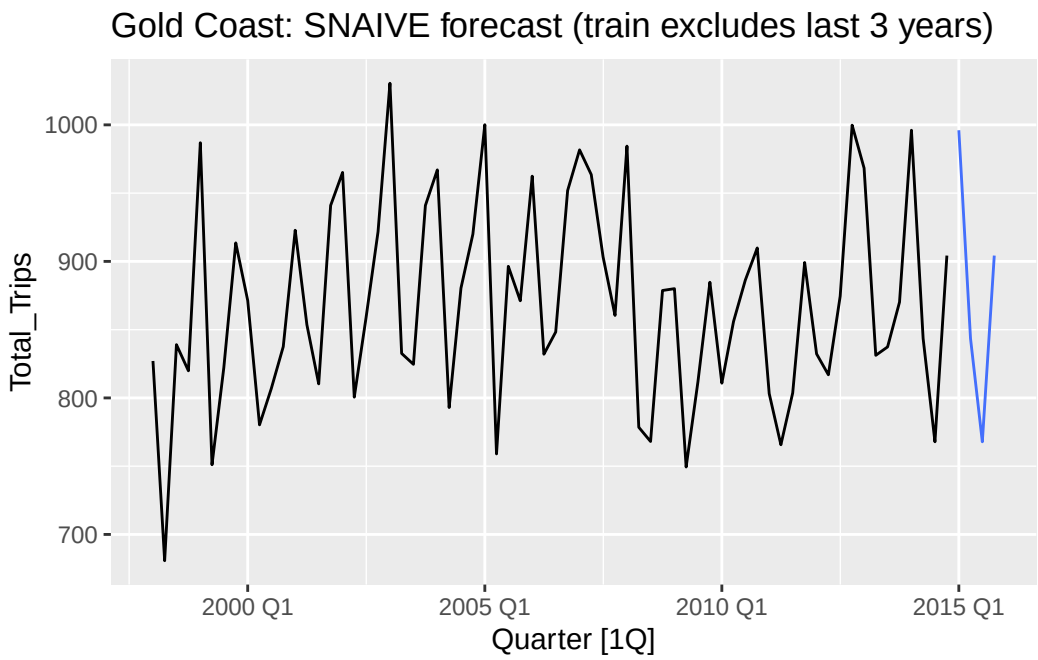
```r
gc_fc_1 <- gc_train_1 |>
  model(SNAIVE = SNAIVE(Total_Trips)) |>
  forecast(h = 4)

autoplot(gc_train_1, Total_Trips) +
  autolayer(gc_fc_1, level = NULL) +
  labs(title = "Gold Coast: SNAIVE forecast (train excludes last 1 year)")
```



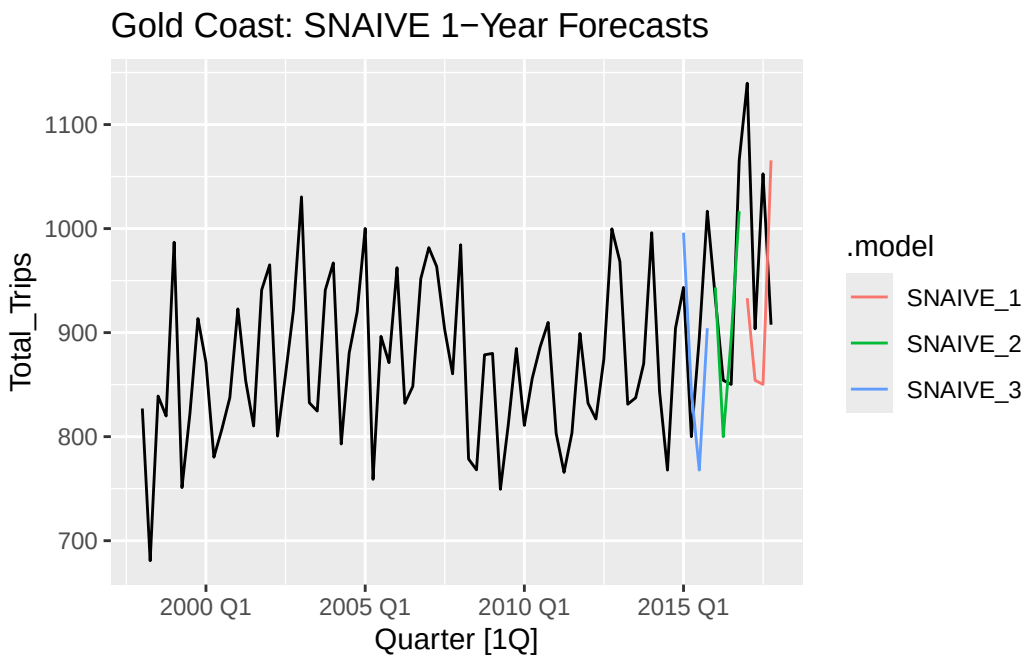

```
gc_fc_2 <- gc_train_2 |>
  model(SNAIVE = SNAIVE(Total_Trips)) |>
  forecast(h = 4)

autoplot(gc_train_2, Total_Trips) +
  autolayer(gc_fc_2, level = NULL) +
  labs(title = "Gold Coast: SNAIVE forecast (train excludes last 2 years)")
```




```
gc_fc_3 <- gc_train_3 |>
  model(SNAIVE = SNAIVE(Total_Trips)) |>
  forecast(h = 4)

autoplot(gc_train_3, Total_Trips) +
  autolayer(gc_fc_3, level = NULL) +
  labs(title = "Gold Coast: SNAIVE forecast (train excludes last 3 years)")
```




```
fit_1 <- gc_train_1 |> model(SNAIVE_1 = SNAIVE(Total_Trips))
fit_2 <- gc_train_2 |> model(SNAIVE_2 = SNAIVE(Total_Trips))
fit_3 <- gc_train_3 |> model(SNAIVE_3 = SNAIVE(Total_Trips))
fits_all <- bind_cols(fit_1, fit_2, fit_3)
gc_fc_all <- fits_all |> forecast(h = 4)
autoplot(gc_tourism, Total_Trips) +
  autolayer(gc_fc_all, level = NULL) +
  labs(title = "Gold Coast: SNAIVE 1-Year Forecasts")
```



d. Use `accuracy()` to compare the test set forecast accuracy using MASE. What can you conclude about the relative performance of the different models? Explain your answer.

```
acc_1 <- accuracy(gc_fc_1, gc_tourism)
acc_2 <- accuracy(gc_fc_2, gc_tourism)
acc_3 <- accuracy(gc_fc_3, gc_tourism)

accuracy_comparison <- bind_rows(
  acc_1 |> mutate(model = "Gold Coast: SNAIVE forecast (train excludes last 1 years)"),
  acc_2 |> mutate(model = "Gold Coast: SNAIVE forecast (train excludes last 2 years)"),
  acc_3 |> mutate(model = "Gold Coast: SNAIVE forecast (train excludes last 3 years)")
) |>
  select(model, MASE) |>
  arrange(MASE)

print(accuracy_comparison)
```

```
# A tibble: 3 x 2
  model                                                      MASE
  <chr>                                                     <dbl>
1 Gold Coast: SNAIVE forecast (train excludes last 2 years) 0.670
2 Gold Coast: SNAIVE forecast (train excludes last 3 years) 1.46
3 Gold Coast: SNAIVE forecast (train excludes last 1 years) 2.66
```

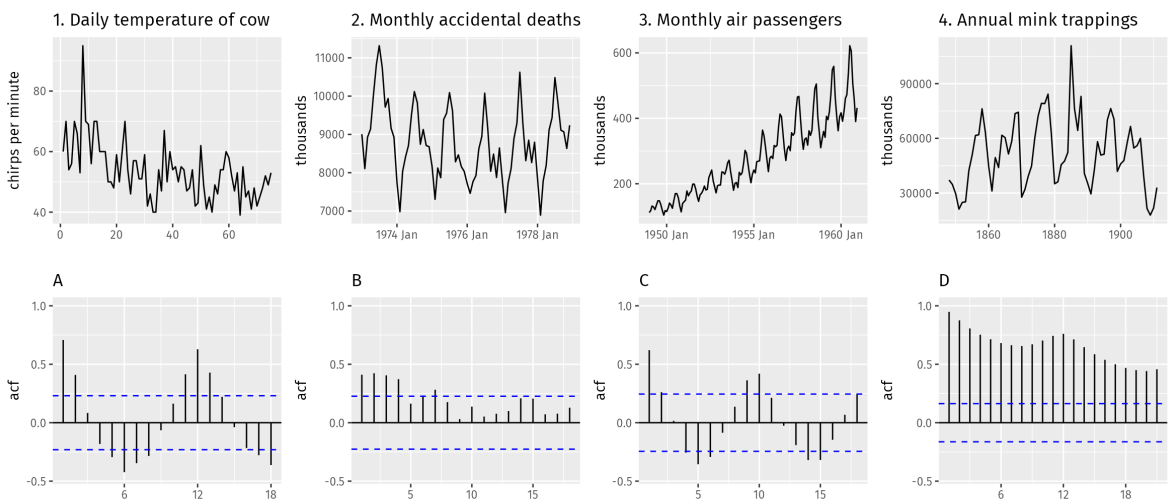
Using the MASE values from the test set accuracy comparison, the model trained by

excluding the last two years performs best, with the lowest MASE (MASE = 0.67). The model excluding the last three years performs moderately (MASE = 1.46), while the model excluding only the last one year performs worst (MASE = 2.66).

Since a lower MASE indicates better forecast accuracy, this suggests that the training window excluding the last two years provides the most reliable seasonal naive forecasts for the test period. The model excluding only one year performs poorly, possibly because the most recent observations differ structurally from earlier patterns, reducing the effectiveness of the seasonal naive assumption.

Question 2 (ACF plots, Q2.9 in FPP)

The following time plots and ACF plots correspond to four different time series. Match each time plot in the first row with one of the ACF plots in the second row.



1. Daily temperature of cow → B
2. Monthly accidental deaths → A
3. Monthly air passengers → D
4. Annual mink trappings → C

Question 3 (Time series classification)

We will investigate this dataset, which contains 600 synthetic control charts for an industrial process. There are 6 types of control charts, and our goal is to build a model to classify them correctly.

We have already processed the raw data into a convenient, labeled form. Run the following code snippet to load it and create train and test sets.

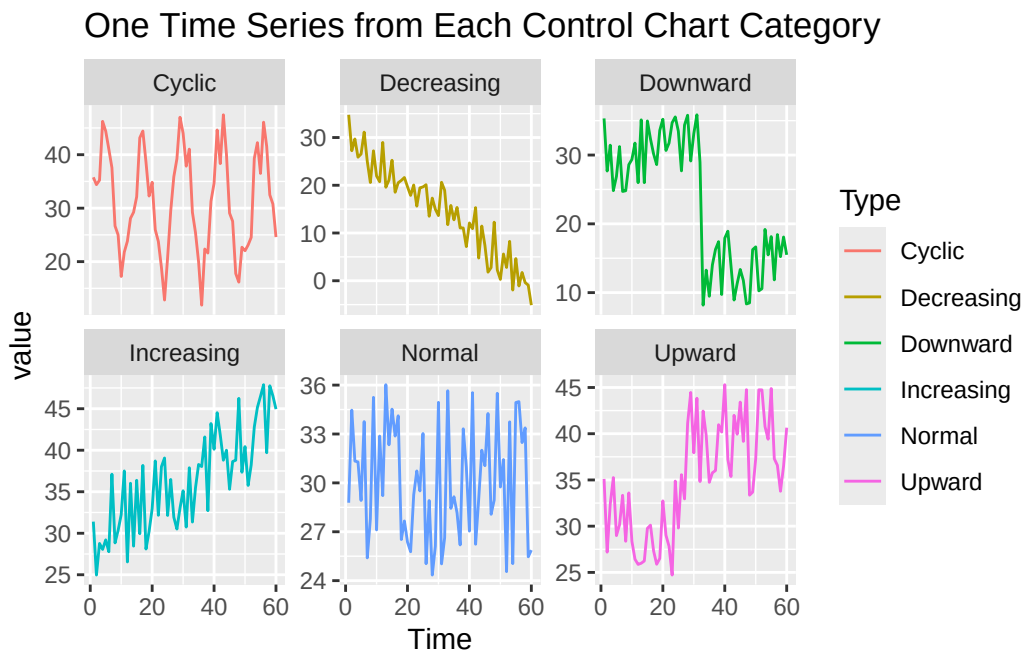
```
ccharts <- read_rds("../_data/CLEANED/ccharts.rds")
ccharts_train <- ccharts[["train"]]
ccharts_test <- ccharts[["test"]]
```

a. Make a time plot of one time series from each category in the training set. Note that the category is recorded under the `Type` column.

```
levels(ccharts_train$Type)
```

```
[1] "Cyclic"     "Decreasing" "Downward"   "Increasing" "Normal"
[6] "Upward"
```

```
one_id_each <- ccharts_train |>
  group_by(Type) |>
  summarise(id = first(id), .groups = "drop")

one_series_each <- ccharts_train |>
  semi_join(one_id_each, by = c("Type", "id"))

ggplot(one_series_each, aes(x = Time, y = value, colour = Type)) +
  geom_line() +
  facet_wrap(~ Type, scales = "free_y") +
  labs(title = "One Time Series from Each Control Chart Category")
```



b. Compute all time series features for both the training and test set using the snippet. What is the difference between `acf1` and `stl_e_acf1` for Increasing, Decreasing, Upward, and Downward types? Why is there a difference?

```
# Change to eval: TRUE in order to run
train_feats <- ccharts_train |> features(value, feature_set(pkgs = "feasts"))
test_feats <- ccharts_test |> features(value, feature_set(pkgs = "feasts"))
```

```
train_feats |>
filter(Type %in% c("Increasing", "Decreasing", "Upward", "Downward")) |>
select(Type, acf1, stl_e_acf1)
```

```
# A tibble: 324 x 3
   Type        acf1 stl_e_acf1
   <fct>      <dbl>      <dbl>
 1 Increasing 0.543    -0.358
 2 Increasing 0.657    -0.190
 3 Increasing 0.814    -0.224
 4 Increasing 0.796    -0.111
 5 Increasing 0.614    -0.0625
 6 Increasing 0.708    -0.226
 7 Increasing 0.782    -0.146
 8 Increasing 0.702    -0.125
```

```
 9 Increasing 0.826    -0.0413
10 Increasing 0.768    -0.292
# i 314 more rows
```

```r
train_feats |>
  filter(Type %in% c("Increasing", "Decreasing", "Upward", "Downward")) |>
  group_by(Type) |>
  summarise(
    mean_acf1 = mean(acf1),
    mean_stl_e_acf1 = mean(stl_e_acf1)
  )
```

```
# A tibble: 4 x 3
  Type       mean_acf1 mean_stl_e_acf1
  <fct>          <dbl>           <dbl>
1 Decreasing     0.695         -0.138
2 Downward       0.711         -0.0755
3 Increasing     0.682         -0.151
4 Upward         0.723         -0.0718
```

For Increasing, Decreasing, Upward, and Downward types, the lag-1 autocorrelation (acf1) is high (around 0.7) because the presence of trend or level shifts induces strong serial dependence between consecutive observations.

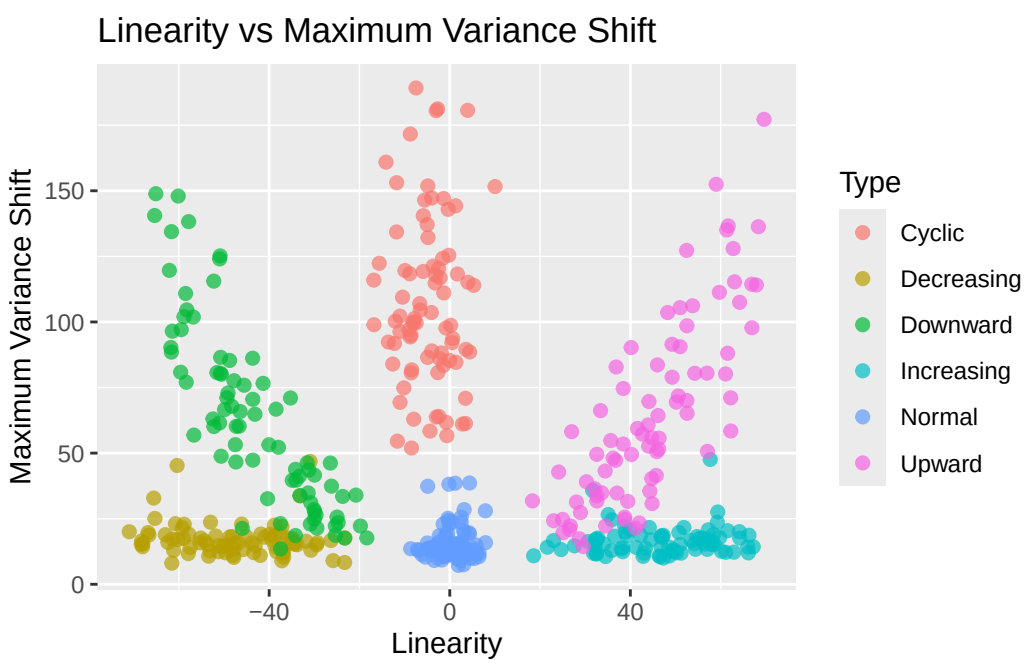
However, after removing the trend using STL decomposition, the autocorrelation of the remainder (stl_e_acf1) becomes small and slightly negative. This difference occurs because trend components artificially inflate autocorrelation in the original series, and once removed, the residual behaves more like weakly correlated noise.

c. Investigate the relationship between the following features and `Type`. Pick two features whose scatter plot gives a good separation between all 6 chart types.

   i. `linearity`
   ii. `trend_strength`
   iii. `acf1`
   iv. `stl_e_acf1`
   v. `shift_level_max`
   vi. `shift_var_max`
   vii. `n_crossing_points`

Make the scatter plot and explain why these features are able to separate the different chart types.

```
train_feats |>
  ggplot(aes(x = linearity, y = shift_var_max, colour = Type)) +
  geom_point(alpha = 0.7, size = 2) +
  labs(
    title = "Linearity vs Maximum Variance Shift",
    x = "Linearity",
    y = "Maximum Variance Shift"
  )
```



The combination of linearity and shift_var_max provides clear separation between the six control chart types. The feature linearity distinguishes monotonic trends: Increasing types exhibit strong positive linearity while Decreasing types exhibit strong negative linearity.

The feature shift_var_max captures structural breaks and variance changes, separating Upward and Downward types from purely trending series. Cyclic charts exhibit low linearity but high variance shifts due to oscillatory behaviour, while Normal charts exhibit both low linearity and low variance shifts. Together, these two features clearly partition the six chart types in the feature space.

d. Install the `caret` package and use the following snippet to fit a k-nearest neighbors model on the two features you have selected (substitute `X` and `Y` for the two features you selected in b), and then predict on the test set. What percentage of the test examples are correctly classified? Write code to compute this value. (You should get $> 90\%$ accuracy)

```r
# Change to eval: TRUE in order to run
library(caret)
```

```
Loading required package: lattice


Attaching package: 'caret'

The following objects are masked from 'package:fabletools':

    MAE, RMSE

The following object is masked from 'package:purrr':

    lift
```

```r
knn_fit <- train(Type ~ linearity + shift_var_max, data = train_feats, method = "knn")
test_pred <- predict(knn_fit, newdata = test_feats)
accuracy <- mean(test_pred == test_feats$Type) * 100
print(accuracy)
```

```
[1] 93.85965
```

```r
confusionMatrix(test_pred, test_feats$Type)
```

```
Confusion Matrix and Statistics

            Reference
Prediction   Cyclic Decreasing Downward Increasing Normal Upward
  Cyclic         18          0        0          0      0      0
  Decreasing      0         17        2          0      0      0
  Downward        0          2       17          0      0      0
  Increasing      0          0        0         19      0      2
  Normal          1          0        0          0     19      0
  Upward          0          0        0          0      0     17

Overall Statistics

               Accuracy : 0.9386
```

```
                 95% CI : (0.8776, 0.975)
    No Information Rate : 0.1667
    P-Value [Acc > NIR] : < 2.2e-16

                  Kappa : 0.9263

 Mcnemar's Test P-Value : NA

Statistics by Class:

                     Class: Cyclic Class: Decreasing Class: Downward
Sensitivity                 0.9474            0.8947          0.8947
Specificity                 1.0000            0.9789          0.9789
Pos Pred Value              1.0000            0.8947          0.8947
Neg Pred Value              0.9896            0.9789          0.9789
Prevalence                  0.1667            0.1667          0.1667
Detection Rate              0.1579            0.1491          0.1491
Detection Prevalence        0.1579            0.1667          0.1667
Balanced Accuracy           0.9737            0.9368          0.9368
                     Class: Increasing Class: Normal Class: Upward
Sensitivity                     1.0000        1.0000        0.8947
Specificity                     0.9789        0.9895        1.0000
Pos Pred Value                  0.9048        0.9500        1.0000
Neg Pred Value                  1.0000        1.0000        0.9794
Prevalence                      0.1667        0.1667        0.1667
Detection Rate                  0.1667        0.1667        0.1491
Detection Prevalence            0.1842        0.1754        0.1491
Balanced Accuracy               0.9895        0.9947        0.9474
```

Question 4 (Periodograms)

Consider the `vic_elec` dataset from the `tsibbledata` package.

```r
vic_elec
```

```
# A tsibble: 52,608 x 5 [30m] <Australia/Melbourne>
   Time                Demand Temperature Date       Holiday
   <dttm>               <dbl>       <dbl> <date>     <lgl>
 1 2012-01-01 00:00:00  4383.        21.4 2012-01-01 TRUE
 2 2012-01-01 00:30:00  4263.        21.0 2012-01-01 TRUE
 3 2012-01-01 01:00:00  4049.        20.7 2012-01-01 TRUE
```

```
 4 2012-01-01 01:30:00  3878.        20.6 2012-01-01 TRUE
 5 2012-01-01 02:00:00  4036.        20.4 2012-01-01 TRUE
 6 2012-01-01 02:30:00  3866.        20.2 2012-01-01 TRUE
 7 2012-01-01 03:00:00  3694.        20.1 2012-01-01 TRUE
 8 2012-01-01 03:30:00  3562.        19.6 2012-01-01 TRUE
 9 2012-01-01 04:00:00  3433.        19.1 2012-01-01 TRUE
10 2012-01-01 04:30:00  3359.        19.0 2012-01-01 TRUE
# i 52,598 more rows
```

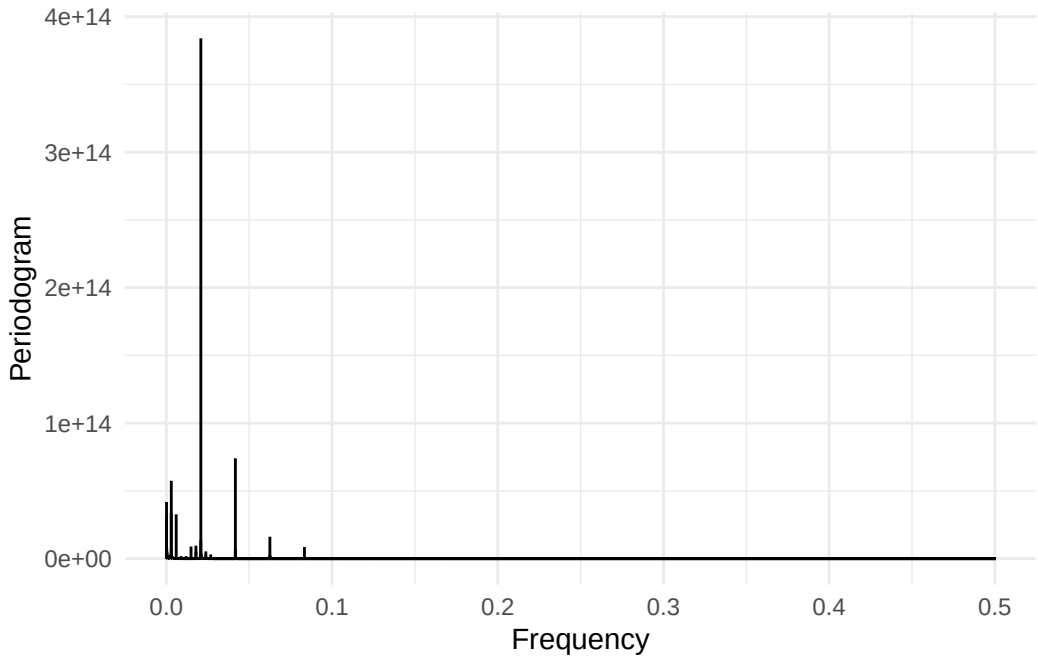
a. Compute and plot the periodogram for `Demand`. What frequencies and periods do the 7 highest peaks correspond to? *Hint: You may use the* `periodogram` *function provided in* `plot_util.R` *and change the* `max_freq` *setting to see at different resolutions.*

```r
source("plot_util.R")
```

```
Loading required package: rlang


Attaching package: 'rlang'

The following objects are masked from 'package:purrr':

    flatten, flatten_chr, flatten_dbl, flatten_int, flatten_lgl,
    flatten_raw, invoke, splice

Loading required package: magrittr


Attaching package: 'magrittr'

The following object is masked from 'package:rlang':

    set_names

The following object is masked from 'package:purrr':

    set_names

The following object is masked from 'package:tidyr':

    extract
```

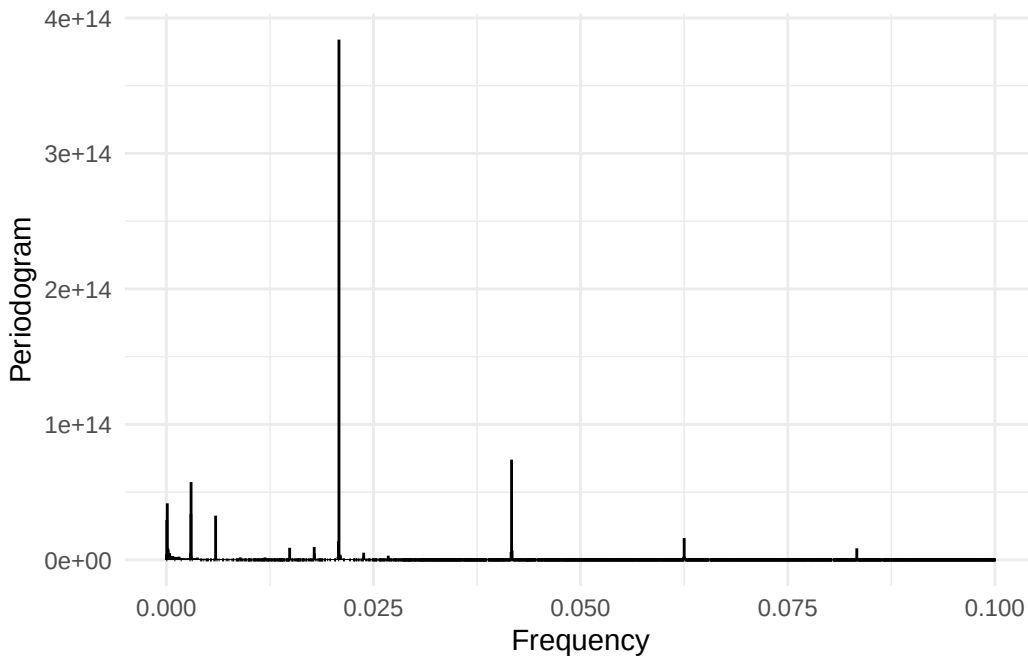
```r
demand <- vic_elec[['Demand']]
periodogram(demand, max_freq = 0.5)
```

```
Warning: Removed 26303 rows containing missing values or values outside the scale range
(`geom_segment()`).
```



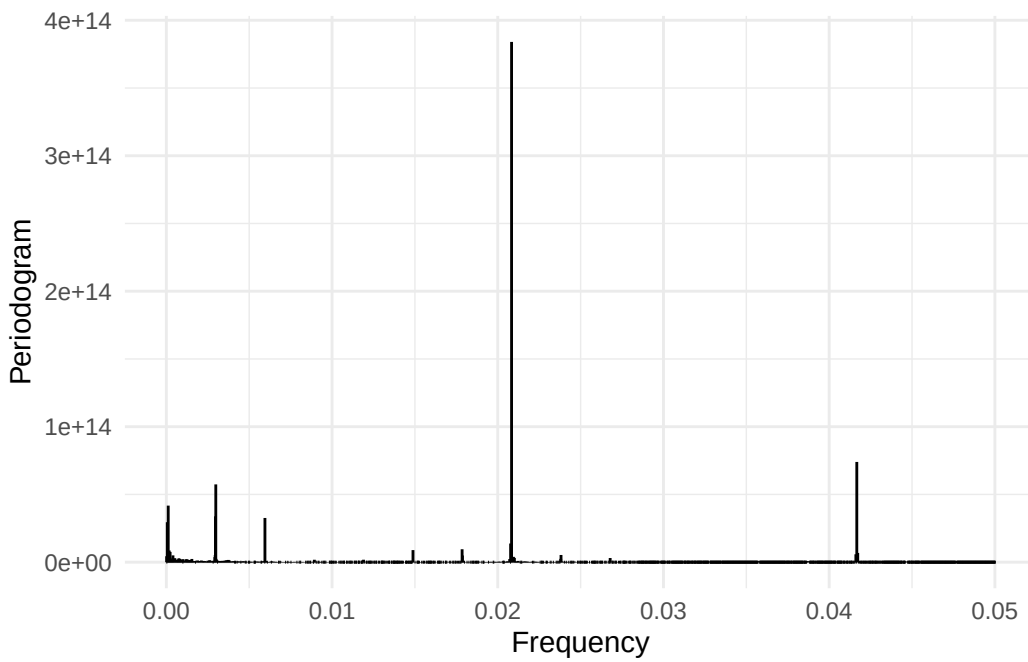

```r
periodogram(demand, max_freq = 0.1)
```

```
Warning: Removed 47347 rows containing missing values or values outside the scale range
(`geom_segment()`).
```



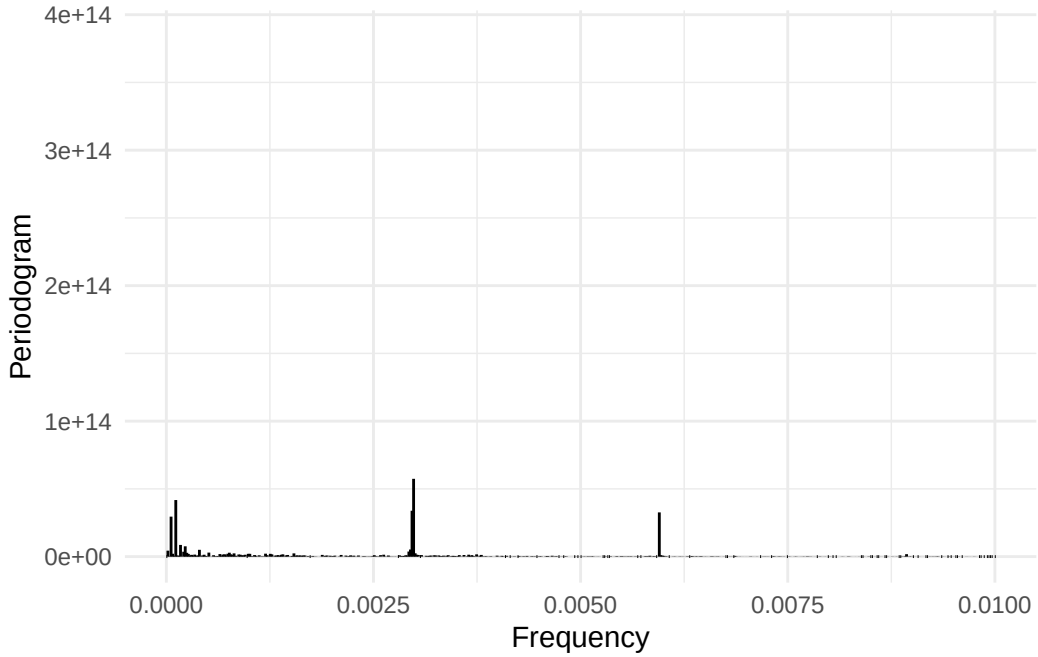

```r
periodogram(demand, max_freq = 0.05)
```

```
Warning: Removed 49977 rows containing missing values or values outside the scale range
(`geom_segment()`).
```



```r
periodogram(demand, max_freq = 0.01)
```

```
Warning: Removed 52081 rows containing missing values or values outside the scale range
(`geom_segment()`).
```




```r
n <- length(demand)
freq <- 0:(n - 1) / n
per  <- Mod(fft(demand - mean(demand, na.rm = TRUE)))^2

period_df <- tibble(freq = freq, per = per) |>
  filter(freq > 0, freq <= 0.5) |>
arrange(desc(per)) |>
head(7) |>
mutate(
period_steps = 1/freq,
period_hours = period_steps * 0.5,
period_days = period_hours / 24,
period_weeks = period_days / 7
)
print(period_df)
```

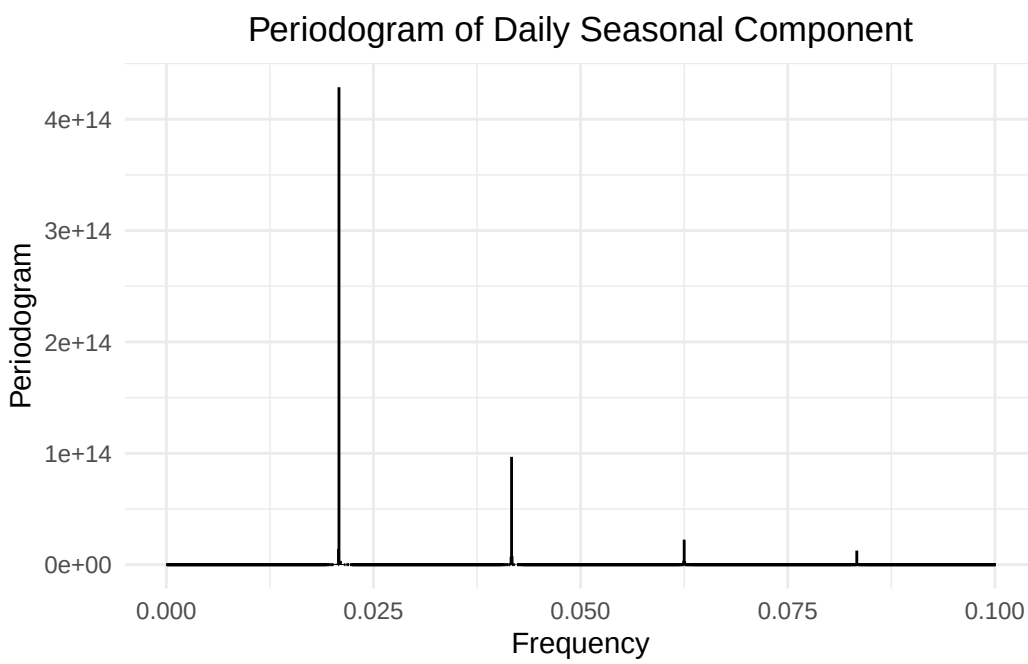
```
# A tibble: 7 x 6
       freq     per period_steps period_hours period_days period_weeks
      <dbl>   <dbl>        <dbl>        <dbl>       <dbl>        <dbl>
```

```
1 0.0208    3.84e14          48          24          1          0.143
2 0.0417    7.40e13          24          12          0.5        0.0714
3 0.00298   5.74e13         335.        168.         6.98       0.997
4 0.000114  4.18e13        8768        4384        183.        26.1
5 0.00297   3.39e13         337.        169.         7.03       1.00
6 0.00595   3.26e13         168.         84.0        3.50       0.500
7 0.0000570 2.95e13       17536        8768        365.        52.2
```

b. Perform an STL decomposition for `Demand`. Plot the periodogram for `season_year`, `season_week`, and `season_day`. Which of the original periodogram peaks appear in each of these periodograms?

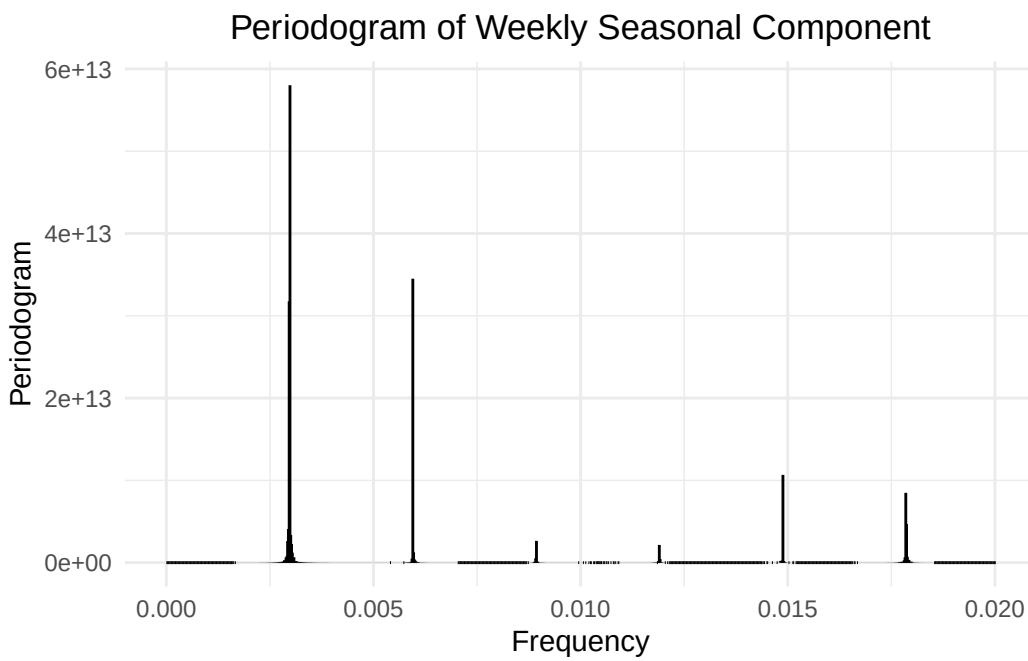
```r
fit <- vic_elec |>
  model(STL(Demand ~ season(period = "year") + season(period = "week") + season(period =

cmp <- components(fit)

periodogram(cmp$season_day, max_freq = 0.10) +
  ggtitle("Periodogram of Daily Seasonal Component") +
  theme(plot.title = element_text(hjust = 0.5))
```

```
Warning: Removed 47347 rows containing missing values or values outside the scale range
(`geom_segment()`).
```



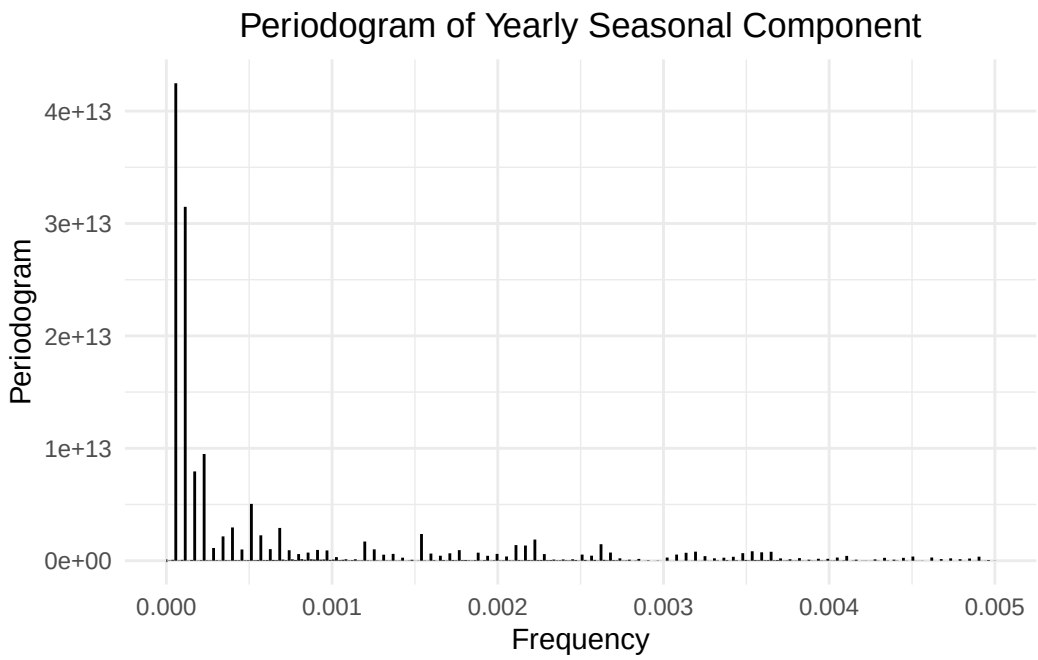
```r
periodogram(cmp$season_week, max_freq = 0.02) +
  ggtitle("Periodogram of Weekly Seasonal Component") +
  theme(plot.title = element_text(hjust = 0.5))
```

```
Warning: Removed 51555 rows containing missing values or values outside the scale range
(`geom_segment()`).
```




```r
periodogram(cmp$season_year, max_freq = 0.005) +
  ggtitle("Periodogram of Yearly Seasonal Component") +
  theme(plot.title = element_text(hjust = 0.5))
```

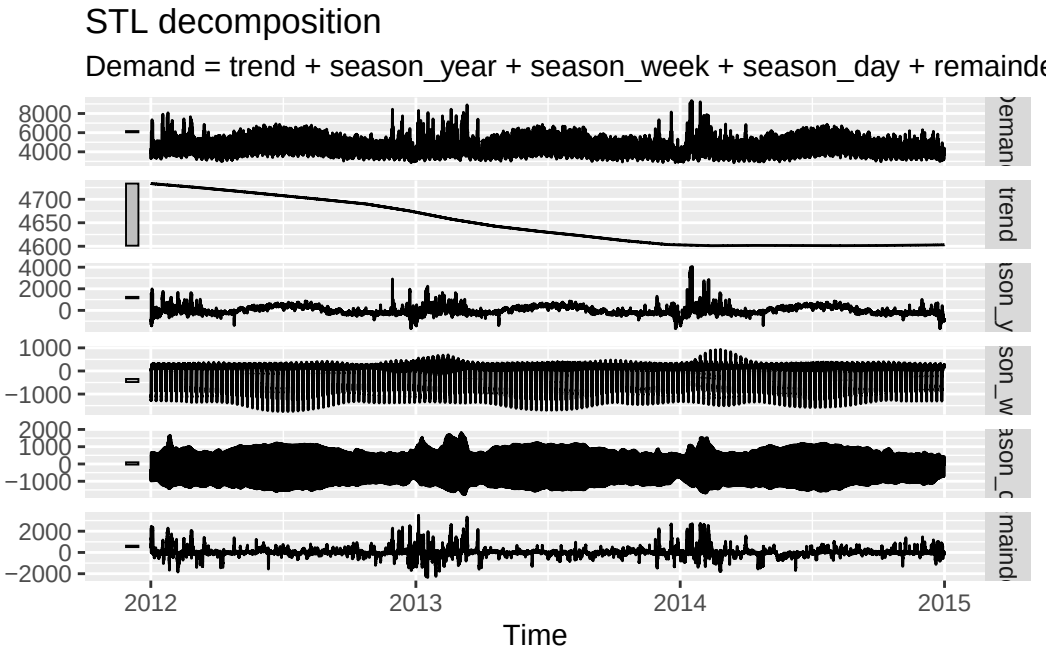
```
Warning: Removed 52344 rows containing missing values or values outside the scale range
(`geom_segment()`).
```



After STL decomposition, the periodogram of season_day contains the dominant daily peak (24 hours) and its harmonics (e.g., 12 hours). The periodogram of season_week contains the weekly peak (7 days) and related harmonics (e.g., ~3.5 days). The periodogram of season_year contains the long-period peaks associated with annual and semi-annual seasonality (~365 and ~182.7 days). Thus, the STL decomposition separates the original spectral peaks according to their seasonal period.

c. Based on the peak heights, which is the strongest seasonal component? Does it agree with what you see in a time series decomposition plot?

```
cmp |> autoplot()
```



The daily seasonal component is strongest because the periodogram has its highest peaks at the daily frequency (24 hours) and its harmonics (e.g., 12 hours). This agrees with the STL decomposition, where the daily seasonal component shows the most prominent repeating pattern relative to the weekly and yearly components.